Установка и настройка Neovim с нуля (Windows 11)

Пошаговая инструкция, как повторить эту конфигурацию на другом компьютере · октябрь 2026

Что получится

- Neovim 0.12 с менеджером плагинов lazy.nvim.
- Подсветка кода (Treesitter), автодополнение (blink.cmp), поиск файлов и текста (Telescope), файловое дерево (nvim-tree), тема tokyonight, значки (Nerd Font).
- Подсказки, ошибки и переход к определению для Python (Pyright), Rust (rust-analyzer) и PowerShell (PowerShell Editor Services).
- Локальный ИИ-помощник: Ollama, модель qwen2.5-coder:7b и плагин codecompanion.nvim.

Нужен интернет. Для локальной модели желательна видеокарта с 8 ГБ памяти: без неё модель работает на процессоре и заметно медленнее. Все команды выполняются в PowerShell. После установки программ закройте PowerShell и откройте заново, чтобы обновились пути.

Markdown и LaTeX описаны отдельно, в инструкции `install-markdown-latex`. Её читайте после этой.

Шаг 1. Установить программы

```
winget install Neovim.Neovim
winget install Git.Git
winget install Python.Python.3.12
winget install OpenJS.NodeJS
winget install BurntSushi.ripgrep.MSVC
winget install tree-sitter.tree-sitter-cli
winget install DEVCOM.JetBrainsMonoNerdFont
winget install Rustlang.Rustup
winget install Ollama.Ollama
```

Заметки:

- Если winget не находит пакет, найдите его командой `winget search имя`: идентификаторы иногда меняются.
- Node.js нужен для Pyright, Python нужен для разбора Python-кода, Rust ставится через rustup.
- Для сборки парсеров Treesitter нужен компилятор C. На исходном компьютере стоит Visual Studio. Если `:TSInstall` пожалуется на отсутствие компилятора, поставьте Build Tools для Visual Studio с компонентом «Разработка классических приложений на C++» или другой компилятор (например, zig).
- Буфер обмена Neovim обычно работает через win32yank, который идёт в комплекте. Проверка: `:checkhealth vim.provider`.

Шаг 2. Выбрать шрифт в Windows Terminal

1. Нажмите `Ctrl+,` (откроются настройки).
2. Слева в разделе «Профили» выберите «По умолчанию», затем вкладку «Оформление».
3. В поле «Шрифт» выберите JetBrainsMono Nerd Font

Neovim setup from scratch (Windows 11)

A step-by-step guide to reproduce this configuration on another computer · October 2026

What you get

- Neovim 0.12 with the lazy.nvim plugin manager.
- Syntax highlighting (Treesitter), autocompletion (blink.cmp), file and text search (Telescope), a file tree (nvim-tree), the tokyonight theme, icons (Nerd Font).
- Hints, errors and go-to-definition for Python (Pyright), Rust (rust-analyzer) and PowerShell (PowerShell Editor Services).
- A local AI assistant: Ollama, the qwen2.5-coder:7b model and the codecompanion.nvim plugin.

You need an internet connection. For the local model a graphics card with 8 GB of memory is recommended: without one the model runs on the CPU and is noticeably slower. All commands are run in PowerShell. After installing programs, close PowerShell and open it again so the paths are updated.

Markdown and LaTeX are covered separately, in the guide `install-markdown-latex`. Read it after this one.

Step 1. Install the programs

```
winget install Neovim.Neovim
winget install Git.Git
winget install Python.Python.3.12
winget install OpenJS.NodeJS
winget install BurntSushi.ripgrep.MSVC
winget install tree-sitter.tree-sitter-cli
winget install DEVCOM.JetBrainsMonoNerdFont
winget install Rustlang.Rustup
winget install Ollama.Ollama
```

Notes:

- If winget cannot find a package, search for it with `winget search name`: identifiers sometimes change.
- Node.js is needed for Pyright, Python is needed for parsing Python code, and Rust is installed via rustup.
- Building Treesitter parsers requires a C compiler. The original computer has Visual Studio. If `:TSInstall` complains about a missing compiler, install Build Tools for Visual Studio with the “Desktop development with C++” workload, or another compiler (for example, zig).
- The Neovim clipboard usually works through win32yank, which is bundled. Check: `:checkhealth vim.provider`.

Step 2. Choose the font in Windows Terminal

1. Press `Ctrl+,` (the settings open).
2. On the left, under “Profiles”, choose “Defaults”, then the “Appearance” tab.
3. In the “Font face” field choose JetBrainsMono Nerd Font

и нажмите «Сохранить».

Если шрифта нет в списке, закройте Windows Terminal полностью и откройте заново. Без этого шага вместо значков будут ромбики.

Шаг 3. Перенести конфиг

Конфиг лежит в папке `$env:LOCALAPPDATA/nvim`. Если она уже есть, переименуйте её:

```
Rename-Item $env:LOCALAPPDATA/nvim nvim.old
```

Вариант А: из GitHub. Команды ниже выполните под текстом.

```
cd $env:LOCALAPPDATA
git clone https://github.com/sliperly/nvim-config-windows-python-rust-powershell-ollama.git nvim
```

Вариант Б: без GitHub. Скопируйте папку `nvim` со старого компьютера (флешка, архив) в `$env:LOCALAPPDATA`. Проверка:

```
Get-ChildItem $env:LOCALAPPDATA/nvim
```

Должны быть `init.lua`, `lua` и `lazy-lock.json`.

Вариант В: вручную. Создайте файлы по приложению в конце этой инструкции.

Файл `lazy-lock.json` закрепляет точные версии плагинов, поэтому на новом компьютере всё ставится ровно таким, как работает сейчас.

Шаг 4. Первый запуск

```
nvim
```

lazy.nvim скачает сам себя и все плагины. Появится окно со списком: подождите, пока загрузка закончится, и закройте окно клавишей `q`. Плагин blink.cmp при первом запуске скачивает готовый бинарный файл (нужен интернет). Если появились красные ошибки, закройте Neovim и запустите заново.

Если конфиг взят из репозитория, в нём уже есть плагины и настройки для Markdown и LaTeX. Они установятся сами. Пока вы не прошли вторую инструкцию, при открытии `.md`- или `.tex`-файла может появиться сообщение, что сервер texlab или marksman не найден. Это нормально.

Шаг 5. Парсеры подсветки

В Neovim выполните:

```
:TSInstall lua python rust
```

Дайте каждому языку 20–40 секунд. Проверка: `:checkhealth nvim-treesitter`. В списке Installed languages должны быть `lua`, `python` и `rust`.

Шаг 6. Языковые серверы

Python (Pyright):

```
npm install -g pyright
pyright --version
```

Rust (rust-analyzer):

```
rustup component add rust-analyzer
rust-analyzer --version
```

and click “Save”.

If the font is not in the list, close Windows Terminal completely and open it again. Without this step you will see diamonds instead of icons.

Step 3. Bring the config over

The config lives in the folder `$env:LOCALAPPDATA/nvim`. If it already exists, rename it:

```
Rename-Item $env:LOCALAPPDATA/nvim nvim.old
```

Option A: from GitHub. Run the commands below the text.

Option B: without GitHub. Copy the `nvim` folder from the old computer (flash drive, archive) into `$env:LOCALAPPDATA`. Check:

```
Get-ChildItem $env:LOCALAPPDATA/nvim
```

You should see `init.lua`, `lua` and `lazy-lock.json`.

Option C: by hand. Create the files from the appendix at the end of this guide.

The file `lazy-lock.json` pins exact plugin versions, so on the new computer everything is installed exactly as it works now.

Step 4. First launch

```
nvim
```

lazy.nvim will download itself and all the plugins. A window with a list appears: wait until loading finishes and close the window with `q`. On first launch the blink.cmp plugin downloads a prebuilt binary (internet needed). If red errors appear, close Neovim and start it again.

If the config was taken from the repository, it already contains plugins and settings for Markdown and LaTeX. They install by themselves. Until you complete the second guide, opening a `.md` or `.tex` file may show a message that the texlab or marksman server was not found. This is normal.

Step 5. Highlighting parsers

In Neovim run:

```
:TSInstall lua python rust
```

Give each language 20–40 seconds. Check: `:checkhealth nvim-treesitter`. The Installed languages list should contain `lua`, `python` and `rust`.

Step 6. Language servers

Python (Pyright):

```
npm install -g pyright
pyright --version
```

Rust (rust-analyzer):

```
rustup component add rust-analyzer
rust-analyzer --version
```

PowerShell (PowerShell Editor Services). Скачать и распаковать (команды ниже, под текстом). Последняя команда должна вывести `True`.

```
$dir = Join-Path $env:LOCALAPPDATA 'nvim-data/pses'
New-Item -ItemType Directory -Force $dir | Out-Null
$url = 'https://github.com/PowerShell/PowerShellEditorServices/releases/latest/download/PowerShellEditorServices.zip'
$zip = Join-Path $dir 'pses.zip'
Invoke-WebRequest $url -OutFile $zip
Expand-Archive $zip -DestinationPath $dir -Force
Test-Path (Join-Path $dir 'PowerShellEditorServices/Start-EditorServices.ps1')
```

Важно: в `lsp.lua` сервер запускается через `powershell` (встроенный Windows PowerShell 5.1), потому что PowerShell 7 из Microsoft Store Neovim запустить не может. Если PowerShell 7 установлен обычным установщиком, можно заменить `powershell` на `pwsh`.

Шаг 7. Локальная модель

Ollama после установки запускается сама (значок ламы в трее). Затем:

```
ollama pull qwen2.5-coder:7b
ollama run qwen2.5-coder:7b 'Hello'
ollama ps
```

Модель занимает около 4,7 ГБ. В колонке PROCESSOR команды `ollama ps` должно быть `100% GPU`. Плагин настроен на контекст 16384 токена, это около 5,5 ГБ видеопамяти. Если видеопамяти меньше 8 ГБ, уменьшите `num_ctx` в `lazy.lua` до 8192.

Шаг 8. Punto Switcher (если установлен)

Punto Switcher переключает раскладку на лету и ломает ввод команд. Правый клик по значку в трее, затем «Настройки», раздел «Программы-исключения», добавить `WindowsTerminal.exe`.

Шаг 9. Проверка

- `:checkhealth` не показывает красных ошибок, а `:checkhealth vim.provider` показывает Python, Node, Perl и Ruby как Disabled. Это отключены внутренние провайдеры Neovim; к программам, установленным в Windows, это отношения не имеет.
- Файл `test.py` с одной строкой `print(abc)`: справа появляется ошибка `abc is not defined`.
- Работают `Ctrl+p` (поиск файла), `Ctrl+f` (поиск текста), `Ctrl+n` (дерево).
- Набор `pri` в Python-файле открывает меню автодополнения.
- Пробел, затем `a` открывает чат, затем `i`, вопрос и `Ctrl+s`: приходит ответ модели.

Приложение. Файлы конфигурации

Папка: `$env:LOCALAPPDATA/nvim`. Код ниже одинаков для обоих языков.

PowerShell (PowerShell Editor Services). Download and unpack (commands below the text). The last command should print `True`.

Important: in `lsp.lua` the server is started through `powershell` (the built-in Windows PowerShell 5.1), because Neovim cannot launch PowerShell 7 from the Microsoft Store. If PowerShell 7 was installed with the regular installer, you can replace `powershell` with `pwsh`.

Step 7. Local model

Ollama starts by itself after installation (a llama icon in the tray). Then:

```
ollama pull qwen2.5-coder:7b
ollama run qwen2.5-coder:7b 'Hello'
ollama ps
```

The model takes about 4.7 GB. In the PROCESSOR column of `ollama ps` you should see `100% GPU`. The plugin is set to a 16384-token context, which is about 5.5 GB of video memory. If you have less than 8 GB of video memory, reduce `num_ctx` in `lazy.lua` to 8192.

Step 8. Punto Switcher (if installed)

Punto Switcher switches the keyboard layout on the fly and breaks command input. Right-click its tray icon, then “Settings”, the “Exception programs” section, and add `WindowsTerminal.exe`.

Step 9. Check

- `:checkhealth` shows no red errors, and `:checkhealth vim.provider` shows Python, Node, Perl and Ruby as Disabled. These are Neovim’s internal providers being switched off; it has nothing to do with programs installed in Windows.
- A file `test.py` with the single line `print(abc)`: an error `abc is not defined` appears on the right.
- `Ctrl+p` (find file), `Ctrl+f` (find text) and `Ctrl+n` (tree) work.
- Typing `pri` in a Python file opens the completion menu.
- Space, then `a` opens the chat, then `i`, a question and `Ctrl+s`: the model replies.

Appendix. Configuration files

Folder: `$env:LOCALAPPDATA/nvim`. The code below is the same for both languages.

init.lua

```
vim.g.loaded_perl_provider = 0
vim.g.loaded_ruby_provider = 0
vim.g.loaded_node_provider = 0
vim.g.loaded_python3_provider = 0

require('config.options')
require('config.lazy')
require('config.lsp')
```

lua/config/options.lua

```
vim.opt.number = true
vim.opt.mouse = "a"
vim.opt.scrolloff = 7
vim.opt.swapfile = false
vim.opt.termguicolors = true
vim.keymap.set('i', 'jk', '<Esc>')
vim.api.nvim_create_autocmd('FileType', {
  pattern = { 'python', 'lua', 'rust' },
  callback = function() pcall(vim.treesitter.start) end,
})
vim.g.mapleader = ' '
vim.keymap.set({ 'n', 'v' }, '<leader>a', '<cmd>CodeCompanionChat Toggle<cr>')
vim.keymap.set({ 'n', 'v' }, '<leader>i', ':CodeCompanion ')
vim.keymap.set('v', '<leader>c', '<cmd>CodeCompanionChat Add<cr>')
```

lua/config/lazy.lua

```
local lazypath = vim.fn.stdpath('data') .. '/lazy/lazy.nvim'
if not (vim.uv or vim.loop).fs_stat(lazypath) then
  vim.fn.system({ 'git', 'clone', '--filter=blob:none', '--branch=stable',
    'https://github.com/folke/lazy.nvim.git', lazypath })
end
vim.opt.rtp:prepend(lazypath)

require('lazy').setup({
  spec = {
    { 'nvim-treesitter/nvim-treesitter', build = ':TSUpdate' },
    { 'saghen/blink.cmp', version = '1.*', opts = {} },
    {
      'nvim-telescope/telescope.nvim',
      dependencies = { 'nvim-lua/plenary.nvim' },
      keys = {
        { '<C-p>', '<cmd>Telescope find_files<cr>' },
        { '<C-f>', '<cmd>Telescope live_grep<cr>' },
      },
    },
    {
      'folke/tokyonight.nvim',
      lazy = false,
      priority = 1000,
      config = function() vim.cmd.colorscheme('tokyonight') end,
    },
    {
      'nvim-tree/nvim-tree.lua',
      dependencies = { 'nvim-tree/nvim-web-devicons' },
      keys = { { '<C-n>', '<cmd>NvimTreeToggle<cr>' } },
      opts = {},
    },
    {
      'olimorris/codecompanion.nvim',
      dependencies = {
        'nvim-lua/plenary.nvim',
        'nvim-treesitter/nvim-treesitter',
      },
      cmd = { 'CodeCompanion', 'CodeCompanionChat', 'CodeCompanionActions' },
      opts = {
        interactions = {
          chat = { adapter = { name = 'ollama', model = 'qwen2.5-coder:7b' } },
          inline = { adapter = { name = 'ollama', model = 'qwen2.5-coder:7b' } },
        },
        adapters = {
          http = {
            ollama = function()
              return require('codecompanion.adapters').extend('ollama', {
                schema = {
                  num_ctx = { default = 16384 },
                  keep_alive = { default = '30m' },
                },
              })
            end,
          },
        },
      },
    },
  },
  rocks = { enabled = false },
})
```

lua/config/lsp.lua

```
vim.lsp.config('pyright', {
  cmd = { 'pyright-langserver', '--stdio' },
  filetypes = { 'python' },
  root_markers = { 'pyrightconfig.json', 'pyproject.toml', 'setup.py', 'requirements.txt', '.git' },
})
vim.lsp.enable('pyright')

vim.diagnostic.config({ virtual_text = true })

vim.api.nvim_create_autocmd('LspAttach', {
  callback = function(args)
    vim.keymap.set('n', 'gd', vim.lsp.buf.definition, { buffer = args.buf })
  end,
})

local pses = vim.fs.normalize(vim.fn.stdpath('data') .. '/pses')
local cache = vim.fs.normalize(vim.fn.stdpath('cache'))
local pses_cmd = string.format(
  "& '%s/PowerShellEditorServices/Start-EditorServices.ps1' "
    .. "-BundledModulesPath '%s' "
    .. "-LogPath '%s/powershell_es.log' "
    .. "-SessionDetailsPath '%s/powershell_es.session.json' "
    .. "-FeatureFlags @() -AdditionalModules @() "
    .. "-HostName nvim -HostProfileId 0 -HostVersion 1.0.0 "
    .. "-Stdio -LogLevel Normal",
  pses, pses, cache, cache
)

vim.lsp.config('powershell_es', {
  cmd = { 'powershell', '-NoLogo', '-ExecutionPolicy', 'Bypass', '-NoProfile', '-Command', pses_cmd },
  filetypes = { 'ps1' },
  root_markers = { 'PSScriptAnalyzerSettings.psd1', '.git' },
})
vim.lsp.enable('powershell_es')

vim.lsp.config('rust_analyzer', {
  cmd = { 'rust-analyzer' },
  filetypes = { 'rust' },
  root_markers = { 'Cargo.toml', 'rust-project.json', '.git' },
})
vim.lsp.enable('rust_analyzer')
```

В репозитории строка запуска PowerShell в `lsp.lua` записана одной длинной строкой. Здесь она разбита на части, чтобы при копировании из PDF не появились лишние переводы строк. Работает она так же.

Резервная копия конфига

Сохранить версию:

```
cd $env:LOCALAPPDATA/nvim
git add .
git commit -m 'Update config'
```

Если папка ещё не репозиторий Git (вариант B), сначала выполните `git init -b master`.

Git подписывает коммиты вашим именем и почтой. Если он их не знает, задайте:

```
git config --global user.name 'YOUR_NAME'
git config --global user.email 'YOUR_EMAIL'
```

Важно: в публичном репозитории почта из коммитов видна всем. На GitHub откройте Settings, затем Emails и включите «Keep my email addresses private». Там же будет скрытый адрес вида `12345678+имя@users.noreply.github.com`: подставьте его вместо `YOUR_EMAIL`. Сделайте это до первого коммита, иначе старые коммиты придётся переписывать.

Выложить на GitHub (по желанию): создайте на github.com пустой репозиторий (закрытый или публичный). Замените в команде `YOUR_REPO_URL` на его адрес.

In the repository the PowerShell launch string in `lsp.lua` is written as one long line. Here it is split into parts so that copying from the PDF does not introduce stray line breaks. It works the same way.

Config backup

Save a version:

```
cd $env:LOCALAPPDATA/nvim
git add .
git commit -m 'Update config'
```

If the folder is not yet a Git repository (option C), first run `git init -b master`.

Git signs commits with your name and email. If it does not know them, set them:

```
git config --global user.name 'YOUR_NAME'
git config --global user.email 'YOUR_EMAIL'
```

Important: in a public repository the email from commits is visible to everyone. On GitHub open Settings, then Emails, and turn on “Keep my email addresses private”. The same page shows a hidden address like `12345678+name@users.noreply.github.com`: use it instead of `YOUR_EMAIL`. Do this before the first commit, otherwise old commits have to be rewritten.

Publish to GitHub (optional): create an empty repository on github.com (private or public). Replace `YOUR_REPO_URL` in the command with its address.

Если конфиг был взят из чужого репозитория (вариант А), адрес origin сейчас ведёт на репозиторий автора. Перенаправьте его на свой:

```
git remote set-url origin YOUR_REPO_URL
git push -u origin master
```

Если origin ещё не задан (варианты Б и В), вместо первой строки выполните `git remote add origin YOUR_REPO_URL`.

В копию попадают только ваши файлы и `lazy-lock.json`. Сами плагины в неё не входят: lazy скачает их при первом запуске. Не кладите в репозиторий пароли и ключи доступа (сейчас их в конфиге нет).

Что дальше

Если нужны Markdown и LaTeX, продолжайте по инструкции `install-markdown-latex`. Шпаргалка по клавишам: `cheatsheet-neovim`.

If the config came from someone else's repository (option A), the origin address now points to the author's repository. Redirect it to your own:

```
git remote set-url origin YOUR_REPO_URL
git push -u origin master
```

If origin is not set yet (options B and C), run `git remote add origin YOUR_REPO_URL` instead of the first line.

Only your files and `lazy-lock.json` go into the copy. The plugins themselves are not included: lazy downloads them on first launch. Do not put passwords or access keys into the repository (there are none in the config now).

What next

If you need Markdown and LaTeX, continue with the guide `install-markdown-latex`. Key reference: `cheatsheet-neovim`.